

Improved UNet for HipMRI Study - 2D and 3D Segmentation

Overview

This project implements both **2D and 3D Improved UNet architectures** for prostate segmentation from MRI data:

- **2D UNet**: Segment 2D slices with 88.93% Dice coefficient
- **3D UNet**: Segment 3D volumetric data with downsampling for memory efficiency

Both models achieve excellent segmentation performance on the HipMRI Study dataset.

Problem Statement and Algorithm

Problem Context

Clinical Challenge: Accurate segmentation of prostate structures from MRI is essential for:

- Clinical diagnosis of prostate cancer
- Treatment planning for radiotherapy
- Surgical guidance
- Longitudinal patient monitoring

Technical Challenge: Class imbalance in medical imaging (background >> foreground), spatial complexity, and computational constraints in 3D analysis.

Solution: Improved UNet Architecture

The **Improved UNet** is an encoder-decoder convolutional neural network designed specifically for medical image segmentation. It addresses the challenges through:

Key Design Features:

1. Encoder-Decoder Architecture with Skip Connections

- **Encoder (Downsampling)**: Progressively reduces spatial dimensions while increasing feature channels ($1 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$)
- **Bottleneck**: Captures global context at the smallest resolution
- **Decoder (Upsampling)**: Restores spatial resolution using transpose convolution
- **Skip Connections**: Concatenate encoder outputs with decoder inputs, preserving fine-grained spatial details lost during downsampling
- **Why**: Enables accurate localization + semantic understanding by combining multi-scale features

2. Batch Normalization

- Normalizes layer inputs (zero mean, unit variance)
- **Benefits**:
 - Stabilizes training → faster convergence
 - Enables higher learning rates
 - Acts as mild regularizer
- **How**: Applied after each convolution before activation

3. Dropout Regularization (50%)

- Randomly disables 50% of activations during training
- **Purpose**: Prevents co-adaptation of neurons, reduces overfitting
- **Effect**: Forces network to learn redundant representations

4. Weighted Dice Loss

- Standard Dice coefficient: $\text{Dice} = \frac{2|X \cap Y|}{|X| + |Y|}$
- With class weights: $\text{Loss} = 1 - \sum_c w_c \cdot \text{Dice}_c$
- **Addresses class imbalance**: Penalizes errors on minority class (prostate)
- **Class Imbalance Problem** (3D): Background voxels $\sim 7\times$ more frequent than prostate voxels
 - Without weighting: Model learns to ignore prostate (24% Dice)
 - With weights [0.25, 1.75]: Model focuses on minority class (93% Dice) → **388% improvement**

2D Architecture:

```

Input (1, H, W)
↓
Encoder L1: Conv 1→64, MaxPool → (64, H/2, W/2)
Encoder L2: Conv 64→128, MaxPool → (128, H/4, W/4)
Encoder L3: Conv 128→256, MaxPool → (256, H/8, W/8)
Encoder L4: Conv 256→512, MaxPool → (512, H/16, W/16)
↓
Bottleneck: Conv 512→512
↓
Decoder L4: UpConv + Skip(L4) → (256, H/8, W/8)
Decoder L3: UpConv + Skip(L3) → (128, H/4, W/4)
Decoder L2: UpConv + Skip(L2) → (64, H/2, W/2)
Decoder L1: UpConv + Skip(L1) → (32, H, W)
↓
Output Conv 32→2 → (2, H, W) [logits for background & prostate]

```

3D Architecture (Memory-Optimized):

```

Input (1, D, H, W)
↓
Encoder L1: Conv3D 1→32, MaxPool3D → (32, D/2, H/2, W/2)
Encoder L2: Conv3D 32→64, MaxPool3D → (64, D/4, H/4, W/4)
Encoder L3: Conv3D 64→128, MaxPool3D → (128, D/8, H/8, W/8)
↓
Bottleneck: Conv3D 128→256
↓
Decoder L3: UpConv3D + Skip(L3) → (128, D/4, H/4, W/4)
Decoder L2: UpConv3D + Skip(L2) → (64, D/2, H/2, W/2)
Decoder L1: UpConv3D + Skip(L1) → (32, D, H, W)
↓
Output Conv3D 32→2 → (2, D, H, W)

```

Why 3D has fewer levels: 3D convolutions are computationally expensive (8× more parameters than 2D). Fewer levels + smaller filters (32 vs 64) + 2× downsampling achieve memory efficiency while preserving accuracy.

Datasets

2D Dataset: HipMRI Study 2D Slices

- **25,320 NiftI files**, 256×128 pixels, 2 classes (background: 0, prostate: 1)

- Pre-split: Train 60% | Validation 20% | Test 20%
- Path: `/home/groups/comp3710/HipMRI_Study_open/keras_slices_data`

3D Dataset: HipMRI Study Volumetric Data

- **38 patients & 211 3D MRI volumes**, variable dimensions, 2 classes
- Downsampled 2x in each dimension for memory efficiency
- Split: Train 60% | Validation 20% | Test 20%
- Paths:
 - Volumes:
`/home/groups/comp3710/HipMRI_Study_open/semantic_MRs`
 - Labels:
`/home/groups/comp3710/HipMRI_Study_open/semantic_labels_only`

Preprocessing

1. **Normalization**: Min-max to [0, 1] for consistent intensity ranges
2. **Augmentation** (2D training only):
 - Random rotation: $\pm 15^\circ$
 - Random horizontal/vertical flip: 50% probability
3. **3D Downsampling**: 2x reduction per dimension using trilinear interpolation
4. **Consistent augmentation**: Same seed for image and label transforms

Justification:

- Normalization improves convergence across different MRI scans
- Augmentation increases dataset diversity and prevents overfitting
- 3D downsampling reduces memory requirements while preserving information
- Train/val/test split (60/20/20) provides robust evaluation

Usage

Installation

```
pip install -r requirements.txt
```

Script Descriptions and Comments

1. `modules.py` - Model Architecture

Purpose: Defines all neural network components

Key Components:

- `ConvBlock` : Double convolution with batch norm and dropout for stable training
- `UpConvBlock` : Transpose convolution for upsampling
- `ImprovedUNet2D` : 2D encoder-decoder with 4 levels
- `ImprovedUNet3D` : 3D encoder-decoder with 3 levels (memory-optimized)
- `DiceLoss` : Weighted Dice loss supporting class imbalance correction
- `dice_coefficient()` : Metric computation for 2D segmentation
- `dice_coefficient_3d()` : Metric computation for 3D segmentation

Inline Comments: Each class includes docstrings explaining architecture, forward pass, and purpose of each component.

2. `dataset.py` - Data Loading

Purpose: Handles loading, preprocessing, and augmentation

Key Classes/Functions:

- `MedicalImageDataset` : PyTorch Dataset for 2D slices with augmentation (rotation $\pm 15^\circ$, flip)
- `VolumetricDataset` : PyTorch Dataset for 3D volumes with $2\times$ downsampling
- `load_keras_slices()` : Load 2D slices with train/val/test split (60/20/20)
- `load_semantic_data()` : Load 3D volumes with downsample factor
- `create_data_loaders()` : Unified wrapper supporting `mode='2d'` or `mode='3d'`

Preprocessing Details:

- Min-max normalization to [0, 1] range
- 2D augmentation: Random rotation, horizontal/vertical flip
- 3D downsampling: Trilinear interpolation for smooth reduction

3. `train.py` - Training Script

Purpose: Training loop with validation, early stopping, and checkpointing

Workflow:

```
1. Parse arguments (--mode, --data_dir, --epochs, --batch_size, etc.)
2. Load data via create_data_loaders()
3. Initialize model (ImprovedUNet2D or ImprovedUNet3D based on mode)
4. Create optimizer (Adam) and loss function (weighted Dice)
5. For each epoch:
    - Train on all batches, compute train loss/Dice
    - Validate, compute val loss/Dice
    - Save checkpoint if val loss improves
    - Early stopping if no improvement for N epochs
6. Plot training curves and save to checkpoints/
```

Key Features:

- Automatic device selection (CUDA if available, else CPU)
- Class weights [0.25, 1.75] applied for 3D to handle 7× class imbalance
- Learning rate scheduling (ReduceLROnPlateau)
- Saves best model and training history as JSON

4. `predict.py` - Inference Script

Purpose: Load trained model and make predictions on new data

Functions:

- `load_model()` : Load checkpoint and initialize model
- `predict_single_image()` : 2D inference (input: H×W → output: class map)
- `predict_single_volume()` : 3D inference with downsampling/upsampling
- `evaluate_predictions()` : Compute Dice score against ground truth
- `visualize_prediction()` : Plot prediction with overlay

Workflow:

1. Load model from checkpoint
2. Load image/volume from NIfTI file
3. Normalize to [0, 1]
4. For 3D: Downsample by factor 2
5. Prepare batch: (1, C, H, W) or (1, C, D, H, W)
6. Forward pass: `model(batch) → logits`
7. Apply softmax → probabilities
8. Argmax → class predictions {0, 1}
9. Compute Dice if ground truth available
10. Visualize and save results

5. `load_nifti.py` - NIfTI Loading Examples

Purpose: Demonstrates Nibabel/Nilearn usage for loading medical imaging data

Functions:

- `load_data_2D()` : Load 2D slices from list of NIfTI files
- `load_data_3D()` : Load 3D volumes from list of NIfTI files
- `to_channels()` : Convert label to one-hot encoding

Usage Context: Reference implementation showing low-level NIfTI handling before integration into PyTorch Dataset.

6. `test.py` , `test2d.py` , `test3d.py` - Automated Testing

Purpose: End-to-end testing for GPU cluster submission

Features:

- Auto-detect data paths (Rangpur
`/home/groups/comp3710/HipMRI_Study_open` or local
`./HipMRI_Study_open`)
- Run training with fixed params (epochs=1, batch_size=1) for quick testing
- Generate validation metrics and visualizations
- Handle missing data gracefully

2D UNet Training (2D Slices)

```
# Automated test script (recommended) - detects data path automatically
python test2d.py

# Manual training with custom parameters
python train.py --mode 2d --data_dir
./HipMRI_Study_open/keras_slices_data --epochs 50 --batch_size 32
```

Expected Output:

- Model checkpoint saved to `checkpoints/best_model_epoch_X.pt`
- Training curves saved to `checkpoints/dice_loss_curves.png`
- Training history (JSON) with epoch-wise metrics
- Dice score ~88% on test set

3D UNet Training (Volumetric Data)

```
# Automated test script (recommended)
python test3d.py

# Manual training with custom parameters
python train.py --mode 3d --data_dir ./HipMRI_Study_open --epochs 50 --
batch_size 1 --downsample_factor 2
```

Expected Output:

- Model checkpoint to `checkpoints_3d/best_model_epoch_X.pt`
- Training metrics with class weights applied
- Dice score ~93% on test set
- Early stopping typically around epoch 2-3

2D Inference

```
import torch
from modules import ImprovedUNet2D
from predict import load_model, predict_single_image

# Load model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = load_model("checkpoints/best_model_epoch_1.pt", mode='2d',
device=device)

# Predict single slice
prediction, image = predict_single_image(
    model,
    "path/to/slice.nii.gz",
    device=device
)
# Returns:
#   prediction shape: (H, W) - class indices {0=background, 1=prostate}
#   image shape: (H, W) - normalized input
```

3D Inference

```
import torch
from modules import ImprovedUNet3D
from predict import load_model, predict_single_volume

# Load model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = load_model("checkpoints_3d/best_model_epoch_1.pt", mode='3d',
device=device)

# Predict volume with downsampling
prediction, downsampled_vol, original_vol = predict_single_volume(
    model,
    "path/to/volume.nii.gz",
    downsample_factor=2,
    device=device
)
# Returns:
#   prediction shape: (D, H, W) - class indices upsampled back to
original
#   downsampled_vol: Volume at 2x downsampled resolution (used by model)
#   original_vol: Original input volume
```

Command-Line Arguments

```
python train.py \  
    --mode 2d                                # or '3d'  
    --data_dir ./HipMRI_Study_open          # path to data  
    --epochs 50                             # number of training epochs  
    --batch_size 32                         # batch size (1-4 for 3D)  
    --learning_rate 0.001                   # Adam learning rate  
    --early_stop_patience 5                 # epochs without improvement  
    --downsample_factor 2                   # 3D only, memory reduction  
    --num_workers 4                         # DataLoader workers
```

Implementation Files

Unified Architecture (consolidated 2D and 3D implementations):

File	Purpose
<code>modules.py</code>	2D/3D components: ConvBlock2D/3D, UpConvBlock2D/3D, ImprovedUNet2D/3D, shared DiceLoss with class weighting
<code>dataset.py</code>	Unified 2D and 3D data loading via <code>mode='2d'/'3d'</code> parameter:
	- 2D: MedicalImageDataset, <code>loadkerasslices()</code> , for 2D slices
	- 3D: VolumetricDataset, <code>loadsemanticdata()</code> , with automatic downsampling (2x)
	- Unified: <code>createdataloaders(mode='2d'/'3d')</code> wrapper function
<code>train.py</code>	Unified training script supporting both 2D and 3D via <code>mode='2d'/'3d'</code> parameter:
	- <code>train(mode='2d'/'3d', ...)</code> : Automatically selects model, loss, and data loader
	- Supports argparse: <code>--mode 2d/3d --epochs 50 --downsample_factor 2</code>
	- Class weights [0.25, 1.75] applied only for 3D to handle imbalance
<code>predict.py</code>	Unified inference script supporting both 2D and 3D via <code>mode='2d'/'3d'</code> parameter:
	- <code>loadmodel(checkpointpath, mode='2d'/'3d')</code> : Load appropriate model
	- <code>predictsingleimage()</code> : 2D inference
	- <code>predictsinglevolume()</code> : 3D inference with downsampling

File	Purpose
	- evaluate_predictions(): Compute Dice scores (auto-selects 2D or 3D function)
<code>load_nifti.py</code>	Example: Loading NIfTI files using nibabel and nilearn
<code>test.py</code>	Automated test suite for 2D Rangpur/local execution (mode='2d')
<code>test2d.py</code>	Alternative test suite for 2D execution
<code>test3d.py</code>	Automated test suite for 3D Rangpur/local execution (mode='3d')

Usage Examples:

```
# 2D Training
python train.py --mode 2d --data_dir
./HipMRI_Study_open/keras_slices_data --epochs 50

# 3D Training
python train.py --mode 3d --data_dir ./HipMRI_Study_open --epochs 50 --
downsample_factor 2

# 2D Prediction
python predict.py --mode 2d --checkpoint ./checkpoints/best_model.pt --
image_path input.nii.gz

# 3D Prediction
python predict.py --mode 3d --checkpoint ./checkpoints_3d/best_model.pt -
-volume_path volume.nii.gz
```

Results

2D UNet Results

Training Configuration: Epochs=1, Batch Size=1, Learning Rate=1e-3, Adam Optimizer

Metric	Value
Training Loss	0.0790
Validation Dice	0.9112
Test Dice (Background)	0.9593
Test Dice (Prostate)	0.8893 ✓
Test Dice (Average)	0.9243

Status: ✓ Exceeds 0.75 target. Prostate segmentation Dice: **88.93%**

3D UNet Results

Training Configuration: Epochs=50 (Early Stopped at Epoch 2), Batch Size=1, Learning Rate=1e-3, Downsample Factor=2x, Early Stopping Patience=2, Adam Optimizer

Class Weights for Loss: [0.25, 1.75] - Applied to handle class imbalance (prostate is ~7x less frequent)

Metric	Value	Status
Training Loss (Best)	0.0896	✓ Converged
Validation Loss (Best)	0.0596	✓ Stable
Best Epoch	2	✓ Early Stop
Validation Dice (Avg)	0.9651	✓ Excellent
Test Dice (Background)	0.9948	✓ Exceeds 0.7
Test Dice (Prostate)	0.9369	✓ Exceeds 0.7
Test Dice (Average)	0.9659	✓ EXCELLENT

Status: ✓✓✓ All metrics far exceed ≥0.7 target. Prostate segmentation Dice: **93.69%** | Background: **99.48%**

Key Improvements Over Initial Attempt:

- Initial attempt (no weights): Class 1 Dice = 0.24 (24%)
- With weighted loss: Class 1 Dice = 0.93 (93%) ✓ **388% improvement**
- Class weighting [0.25, 1.75] penalizes underrepresented class more
- Training with 50 epochs allows proper convergence

3D-specific Implementation Details:

- **Input:** 3D volumetric MRI (D, H, W) instead of 2D slices
- **Downsampling:** 2x reduction per dimension → ~8x memory reduction (2^3)
- **Architecture:** 3 encoder/decoder levels (vs 4 for 2D) to fit in GPU memory
- **Filters:** 32 base filters (vs 64 for 2D) for memory efficiency
- **Interpolation:** Trilinear for upsampling and spatial matching
- **Data handling:** VolumetricDataset with caching for efficient loading
- **Loss function:** Weighted Dice Loss with class weights [0.25, 1.75] to handle severe class imbalance

Example Output

Input: MRI slice (256×128, single-channel grayscale) **Output:** Segmentation mask (256×128, two classes)

Visualizations saved to `prediction_results/` :

- `test_sample_000.png` - Side-by-side input/prediction/ground-truth
- `test_predictions_grid.png` - Overview grid of all predictions
- `dice_distribution.png` - Histogram of Dice scores per class

Training metrics saved to `checkpoints/` :

- `dice_loss_curves.png` - 4-panel plot: loss, Dice per epoch, per-class Dice
- `training_history.json` - Numerical results

Resource Optimization

Storage Constraint Solution (10GB limit):

- Removed unnecessary packages: TensorFlow, Keras, scipy, scikit-image, opencv-python, pandas, nilearn

- Final dependencies: torch, torchvision, nibabel, numpy, matplotlib, tqdm (~2-3GB)
- Training: 1 epoch, batch size 1 → minimal memory, excellent quality
- **3D optimization**: Downsampling (2x) reduces memory footprint by ~8x per volume

References

1. Ronneberger et al. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.
 2. Dice, L. R. (1945). Measures of Ecologic Association Between Species. Ecology.
 3. Milletari et al. (2016). V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. 3DV.
-

Author: Jun Xi Koh (48225421) | **Course:** COMP3710 | **Date:** 2025-10-27 | **Version:** 4.0
(2D + 3D UNet)